

Desenvolvedor(a) em IA para Análise Preditiva [T1]

Mini-Projeto Avaliativo - Módulo 1 - Semana 08

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. RESULTADOS ESPERADOS (ENTREGA)	3
4. REQUISITOS DAS TAREFAS	3
4.1. ORGANIZAÇÃO DO PROJETO	3
4.2. REQUISITOS FUNCIONAIS	4
RF01 – Criar ou Carregar o Dataset de Vendas	4
RF02 – Inspecionar e Descrever os Dados	6
RF03 – Limpar e Tratar os Dados	7
RF04 – Criar Colunas Derivadas com Transformações	8
RF05 – Calcular Métricas Agregadas (groupby)	9
RF06 – Segmentar Clientes por Nível de Gasto	11
RF07 – Calcular Estatísticas com NumPy	12
RF08 – Criar Visualizações com Matplotlib e Seaborn	13
RF09 – Criar uma Classe para o Pipeline	15
RF10 – Usar Herança	17
RF11 – Usar Funções Lambda e Funções de Ordem Superior	19
RF12 – Ler e Escrever Arquivos (CSV e JSON)	20
RF13 – Usar Expressões Regulares para Limpeza de Dados	21
RF14 – Executar o Pipeline Completo (Ponto de Entrada)	22
4.3. REQUISITOS TÉCNICOS	23
4.4. ORGANIZAÇÃO DO KANBAN	25
4.5. VERSIONAMENTO COM GITHUB	26
4.6. GRAVAÇÃO DE VÍDEO	27
5. CRITÉRIOS DE AVALIAÇÃO	28
6. CHECKLIST FINAL DE ENTREGA	29
7. MODELO DE README.md (SUGESTÃO)	30

1. CONTEXTUALIZAÇÃO

O mercado de Inteligência Artificial e Ciência de Dados exige que você saiba demonstrar raciocínio lógico, organização, capacidade de resolver problemas com dados reais e boa comunicação técnica. Mesmo antes de construir modelos preditivos avançados com redes

neurais ou aprendizado de máquina, já é possível construir um projeto completo e útil para seu portfólio usando Python, Pandas, NumPy e visualizações.

Neste mini-projeto, você deverá criar um analisador de dados de vendas com pipeline de análise preditiva simples, aproximando você de uma situação real: coletar, limpar, transformar e visualizar dados, identificar padrões e gerar insights acionáveis a partir de um dataset.

Este mini-projeto trabalha diretamente os conteúdos estudados no módulo: lógica de programação com Python; variáveis e tipos de dados; operadores e condicionais; estruturas de repetição; funções e modularização; funções lambda; leitura e escrita de arquivos (CSV, JSON); manipulação de datas com datetime; expressões regulares básicas; Pandas (Series, DataFrames, filtros, groupby, merge, pivot); NumPy (arrays, operações vetorizadas, broadcasting); transformações condicionais; visualização com Matplotlib e Seaborn; e organização de código em classes e módulos.

O projeto também reforça sua organização profissional usando o Kanban para visualizar e acompanhar o progresso das tarefas, e o GitHub Flow com branches descritivas, commits com mensagens claras e histórico que reflete a evolução lógica do trabalho.

2. DESAFIO

Você acaba de ser contratado como Analista de Dados Júnior em uma empresa de varejo que possui um histórico de vendas em formato CSV e precisa urgentemente de um relatório analítico para a reunião trimestral da diretoria. O time de negócios quer entender:

- Como as vendas se comportam ao longo do tempo (por mês, por trimestre);
- Quais produtos e categorias geram mais receita;
- Quais regiões têm melhor desempenho;
- Quais clientes são mais valiosos (segmentação simples);
- Uma projeção simples de tendência de receita para o próximo período.

Você deverá desenvolver um mini-projeto em Python chamado:

SalesInsight PY: Pipeline de Análise e Visualização de Dados de Vendas

O objetivo do projeto é criar um pipeline de análise de dados completo que leia, limpe, transforme, analise e visualize um dataset de vendas, gerando insights relevantes e um relatório exportado.

A aplicação deverá:

- Ler e inspecionar os dados brutos de um arquivo CSV;
- Limpar e tratar dados inconsistentes (valores nulos, formatos de data errados, espaços extras em strings);
- Criar novas colunas derivadas (receita total por linha, mês, trimestre, ano);
- Calcular métricas agregadas por produto, categoria e região;

- Segmentar clientes por nível de gasto (baixo, médio, alto);
- Gerar visualizações claras e informativas;
- Exportar um relatório resumido em CSV e um gráfico em PNG;
- Organizar todo o pipeline em funções reutilizáveis e ao menos uma classe.

Maneira de Execução

O projeto poderá ser executado de uma destas formas:

- Google Colab (recomendado para a disciplina): basta fazer upload do arquivo .py ou .ipynb e rodar as células;
- VS Code com extensão Python e o arquivo .py rodando localmente via terminal;
- Terminal/Prompt de Comando: `python salesinsight.py` (requer Python instalado e bibliotecas instaladas via pip).

O Google Colab já possui Pandas, NumPy, Matplotlib e Seaborn instalados. Para execução local, será necessário instalar as dependências com `pip install pandas numpy matplotlib seaborn`.

3. RESULTADOS ESPERADOS (ENTREGA)

Ao final do mini-projeto, o estudante ou squad (pequeno grupo de até 03 pessoas) deverá entregar:

- Um repositório público no GitHub com o projeto.
- Um arquivo Python principal chamado `salesinsight.py` (e/ou um notebook `salesinsight.ipynb`).
- O arquivo de dataset utilizado (`vendas.csv`) ou as instruções para gerá-lo (caso seja gerado sinteticamente no próprio código).
- Um arquivo `README.md` explicando o projeto.
- Um quadro Kanban com as tarefas realizadas.
- Um histórico de commits no GitHub.
- Um vídeo de até 5 minutos demonstrando o funcionamento.
- Os links submetidos no AVA.

Entrega no AVA

A entrega deverá ser submetida na tarefa do AVA: "Módulo 01 – Mini-Projeto Avaliativo" Prazo: **08/06/2026 até às 12h** Peso do mini-projeto na nota: Avaliação M1.1 – 25% da nota do módulo

Links obrigatórios

O estudante deverá enviar no AVA:

- Link do repositório público no GitHub;

- Link do quadro Kanban utilizado no projeto.

4. REQUISITOS DAS TAREFAS

4.1. ORGANIZAÇÃO DO PROJETO

A estrutura mínima do projeto deverá ser:

Python

```
salesinsight-py/  
|  
├─ salesinsight.py  
├─ vendas.csv  
└─ README.md
```

Para squads (grupos de até ~03 alunos) que quiserem organizar melhor, será aceita a seguinte estrutura:

Python

```
salesinsight-py/  
|  
├─ README.md  
├─ salesinsight.py  
├─ vendas.csv  
├─ outputs/  
|   ├─ relatorio_resumo.csv  
|   └─ graficos/  
|       ├─ vendas_por_mes.png  
|       ├─ top_produtos.png  
|       └─ distribuicao_regioes.png  
└─ planejamento/  
    └─ tarefas-kanban.md
```

Não é necessário criar todas as pastas. O foco é demonstrar domínio dos conceitos de Python e análise de dados estudados até a Semana 06.

4.2. REQUISITOS FUNCIONAIS

RF01 – Criar ou Carregar o Dataset de Vendas

O projeto deverá ter um dataset de vendas. O estudante pode **gerar o dataset sinteticamente** no próprio código (recomendado para facilitar a reprodução) ou utilizar um arquivo CSV real.

Opção A – Gerar o dataset no código (recomendado):

Python

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import random

def gerar_dataset_vendas(n_registros=200, seed=42):
    """Gera um dataset sintético de vendas com dados intencionalmente
    sujos."""
    random.seed(seed)
    np.random.seed(seed)

    produtos = ["Notebook", "Smartphone", "Tablet", "Monitor", "Teclado",
                "Mouse", "Headset"]
    categorias = {"Notebook": "Computadores", "Smartphone": "Celulares",
                 "Tablet": "Celulares",
                 "Monitor": "Computadores", "Teclado": "Periféricos",
                 "Mouse": "Periféricos",
                 "Headset": "Periféricos"}
    regioes = ["Sudeste", "Sul", "Nordeste", "Centro-Oeste", "Norte"]
    clientes = [f"Cliente_{i:03d}" for i in range(1, 51)]

    data_inicio = datetime(2024, 1, 1)
    dados = []

    for i in range(n_registros):
        produto = random.choice(produtos)
        quantidade = random.randint(1, 10)
        preco_base = {"Notebook": 3500, "Smartphone": 2200, "Tablet":
1800,
                    "Monitor": 1200, "Teclado": 250, "Mouse": 120,
                    "Headset": 350}[produto]
```

```
preco = round(preco_base * random.uniform(0.85, 1.15), 2)
data = data_inicio + timedelta(days=random.randint(0, 364))

# Inserindo dados intencionalmente sujos para limpeza
if random.random() < 0.05:
    quantidade = None          # valor nulo
if random.random() < 0.04:
    preco = None               # valor nulo
if random.random() < 0.03:
    produto = " " + produto   # espaço extra (string suja)

dados.append({
    "id_venda": i + 1,
    "data_venda": data.strftime("%Y-%m-%d") if random.random() >
0.02 else "DATA INVÁLIDA",
    "cliente": random.choice(clientes),
    "produto": produto,
    "categoria": categorias.get(produto.strip(), "Outros"),
    "regiao": random.choice(regioes),
    "quantidade": quantidade,
    "preco_unitario": preco
})

return pd.DataFrame(dados)

# Gerar e salvar
df_bruto = gerar_dataset_vendas()
df_bruto.to_csv("vendas.csv", index=False)
print(f"Dataset gerado com {len(df_bruto)} registros.")
print(df_bruto.head())
```

Opção B – Usar um CSV externo: O estudante pode baixar um dataset público (ex.: Kaggle, dados.gov.br) e adaptá-lo ao projeto, desde que contenha ao menos as colunas: data, produto, categoria, quantidade, preço e região.

RF02 – Inspeccionar e Descrever os Dados

O projeto deverá inspecionar o dataset carregado, exibindo informações básicas. Exemplo de resultado esperado no console:

Python

```
=== INSPEÇÃO INICIAL DO DATASET ===
Shape: (200, 8)
Colunas: ['id_venda', 'data_venda', 'cliente', 'produto', 'categoria',
'regiao', 'quantidade', 'preco_unitario']
Tipos de dados:
id_venda          int64
data_venda        object
cliente           object
...
Valores nulos por coluna:
quantidade        10
preco_unitario     8
...
Primeiros registros:
id_venda  data_venda  cliente  produto ...
```

Exemplo de função:

Python

```
def inspecionar_dados(df):
    """Exibe informações básicas do DataFrame."""
    print("\n=== INSPEÇÃO INICIAL DO DATASET ===")
    print(f"Shape: {df.shape}")
    print(f"\nColunas: {list(df.columns)}")
    print(f"\nTipos de dados:\n{df.dtypes}")
    print(f"\nValores nulos por coluna:\n{df.isnull().sum()}")
    print(f"\nPrimeiros registros:\n{df.head()}")
    print(f"\nEstatísticas descritivas:\n{df.describe()}")
```

RF03 – Limpar e Tratar os Dados

O projeto deverá realizar a limpeza dos dados. O estudante deve tratar ao menos os seguintes problemas:

- Remover ou imputar linhas com valores nulos nas colunas críticas (quantidade, preco_unitario);
- Remover linhas com datas inválidas (ex.: "DATA INVÁLIDA");
- Converter a coluna de data para o tipo datetime;
- Remover espaços extras em colunas de texto com `.str.strip()`;
- Registrar no console quantos registros foram removidos.

Exemplo:

Python

```
import re

def limpar_dados(df):
    """
    Limpa e trata o DataFrame de vendas.
    Retorna o DataFrame limpo e um relatório de limpeza.
    """
    n_inicial = len(df)
    relatorio = {}

    # 1. Remover espaços extras em colunas de texto
    colunas_texto = df.select_dtypes(include="object").columns
    for col in colunas_texto:
        df[col] = df[col].str.strip()

    # 2. Converter data e remover datas inválidas
    df["data_venda"] = pd.to_datetime(df["data_venda"], errors="coerce")
    n_dados_invalidas = df["data_venda"].isnull().sum()
    df = df.dropna(subset=["data_venda"])
    relatorio["datas_invalidas_removidas"] = n_dados_invalidas

    # 3. Remover linhas com quantidade ou preço nulos
    n_antes = len(df)
    df = df.dropna(subset=["quantidade", "preco_unitario"])
    relatorio["linhas_nulas_removidas"] = n_antes - len(df)

    # 4. Garantir tipos numéricos corretos
    df["quantidade"] = df["quantidade"].astype(int)
    df["preco_unitario"] = df["preco_unitario"].astype(float)

    n_final = len(df)
    relatorio["registros_iniciais"] = n_inicial
    relatorio["registros_finais"] = n_final
    relatorio["registros_removidos_total"] = n_inicial - n_final

    print("\n=== RELATÓRIO DE LIMPEZA ===")
    for chave, valor in relatorio.items():
        print(f"  {chave}: {valor}")
```



```
return df, relatorio
```

RF04 – Criar Colunas Derivadas com Transformações

O projeto deverá criar novas colunas a partir dos dados existentes, enriquecendo o dataset para análise. Deve-se criar ao menos:

- receita_total: quantidade * preco_unitario
- mes: mês extraído da data (número ou nome)
- trimestre: trimestre do ano (Q1, Q2, Q3, Q4)
- ano: ano extraído da data
- faixa_receita_item: classificação da receita por item usando transformação condicional

Exemplo:

Python

```
def criar_colunas_derivadas(df):  
    """Cria colunas calculadas e derivadas a partir do dataset limpo."""  
  
    # Receita total por linha de venda  
    df["receita_total"] = df["quantidade"] * df["preco_unitario"]  
  
    # Extração de componentes de data  
    df["mes"] = df["data_venda"].dt.month  
    df["mes_nome"] = df["data_venda"].dt.strftime("%B") # nome do mês  
    df["trimestre"] = df["data_venda"].dt.quarter.apply(lambda q: f"Q{q}")  
    df["ano"] = df["data_venda"].dt.year  
  
    # Classificação da receita por item com numpy.select (transformação  
    condicional vetorizada)  
    condicoes = [  
        df["receita_total"] < 500,  
        (df["receita_total"] >= 500) & (df["receita_total"] < 5000),  
        df["receita_total"] >= 5000  
    ]  
    classificacoes = ["Baixo Valor", "Médio Valor", "Alto Valor"]  
    df["faixa_receita_item"] = np.select(condicoes, classificacoes,  
    default="Não Classificado")
```

```
print("\n=== COLUNAS DERIVADAS CRIADAS ===")
print(df[["data_venda", "receita_total", "mes", "trimestre",
"faixa_receita_item"]].head())

return df
```

RF05 – Calcular Métricas Agregadas (groupby)

O sistema deverá calcular e exibir métricas agrupadas por diferentes dimensões. Deve-se gerar ao menos:

- Receita total e quantidade vendida **por mês**;
- Receita total **por produto** (top 5);
- Receita total **por categoria**;
- Receita total **por região**.

Exemplo de resultado esperado no console:

Python

```
=== RECEITA POR MÊS ===
   mes  receita_total  quantidade
1    1      45230.50         312
2    2      38910.00         289
...

=== TOP 5 PRODUTOS POR RECEITA ===
   produto  receita_total
1  Notebook      125430.00
2  Smartphone      98320.50
...
```

Exemplo de implementação:

Python

```
def calcular_metricas(df):
    """Calcula e retorna métricas agregadas do dataset."""
    metricas = {}

    # Receita por mês
    por_mes = df.groupby("mes").agg(
```

```

        receita_total=("receita_total", "sum"),
        quantidade=("quantidade", "sum"),
        n_vendas=("id_venda", "count")
    ).reset_index().sort_values("mes")
    metricas["por_mes"] = por_mes

# Top 5 produtos por receita
top_produtos = df.groupby("produto")["receita_total"].sum()\
                .sort_values(ascending=False).head(5).reset_index()
metricas["top_produtos"] = top_produtos

# Receita por categoria
por_categoria =
df.groupby("categoria")["receita_total"].sum().reset_index()
metricas["por_categoria"] = por_categoria

# Receita por região
por_regiao = df.groupby("regiao").agg(
    receita_total=("receita_total", "sum"),
    media_ticket=("receita_total", "mean")
).reset_index().sort_values("receita_total", ascending=False)
metricas["por_regiao"] = por_regiao

# Exibição
for nome, tabela in metricas.items():
    print(f"\n=== {nome.upper().replace('_', ' ')} ===")
    print(tabela.to_string(index=False))

return metricas

```

RF06 – Segmentar Clientes por Nível de Gasto

O sistema deverá agrupar os dados por cliente, calcular o total gasto por cada um e classificá-los em segmentos. Deve-se usar ao menos uma função lambda e uma transformação condicional.

Critério sugerido de segmentação:

Gasto Total	Segmento
-------------	----------

Abaixo de R\$ 5.000	Bronze
R\$ 5.000 a R\$ 15.000	Prata
Acima de R\$ 15.000	Ouro

Exemplo:

Python

```
def segmentar_clientes(df):  
    """Segmenta clientes pelo total gasto usando groupby e lambda."""  
  
    clientes = df.groupby("cliente")["receita_total"].sum().reset_index()  
    clientes.columns = ["cliente", "total_gasto"]  
  
    # Classificação usando função lambda com condicionais  
    clientes["segmento"] = clientes["total_gasto"].apply(  
        lambda gasto: "Ouro" if gasto > 15000  
        else ("Prata" if gasto >= 5000 else "Bronze")  
    )  
  
    clientes = clientes.sort_values("total_gasto", ascending=False)  
  
    print("\n=== SEGMENTAÇÃO DE CLIENTES ===")  
    print(clientes.head(10).to_string(index=False))  
    print(f"\nDistribuição de  
segmentos:\n{clientes['segmento'].value_counts()}")  
  
    return clientes
```

RF07 – Calcular Estatísticas com NumPy

O projeto deverá usar NumPy diretamente para calcular estatísticas sobre os dados. Deve-se demonstrar ao menos:

- Conversão de uma coluna do DataFrame para array NumPy;
- Uso de operações vetorizadas (sem loops);
- Uso de broadcasting ou operações entre arrays;
- Pelo menos 3 funções NumPy ([mean](#), [std](#), [median](#), [percentile](#), [sum](#), etc.).

Exemplo:

Python

```
def calcular_estatisticas_numpy(df):
    """Usa NumPy para calcular estatísticas sobre as receitas."""
    print("\n=== ESTATÍSTICAS COM NUMPY ===")

    receitas = df["receita_total"].to_numpy() # Converte para array NumPy

    media = np.mean(receitas)
    mediana = np.median(receitas)
    desvio_padrao = np.std(receitas)
    total = np.sum(receitas)
    p25 = np.percentile(receitas, 25)
    p75 = np.percentile(receitas, 75)

    print(f"  Receita média por venda:      R$ {media:.2f}")
    print(f"  Receita mediana por venda:    R$ {mediana:.2f}")
    print(f"  Desvio padrão:                  R$ {desvio_padrao:.2f}")
    print(f"  Receita total:                  R$ {total:.2f}")
    print(f"  Percentil 25 (Q1):             R$ {p25:.2f}")
    print(f"  Percentil 75 (Q3):            R$ {p75:.2f}")

    # Broadcasting: normalizar receitas entre 0 e 1
    receitas_normalizadas = (receitas - receitas.min()) / (receitas.max()
- receitas.min())
    print(f"\n  Receitas normalizadas (primeiros 5):
{receitas_normalizadas[:5].round(4)}")

    # Operação vetorizada: identificar vendas acima da média sem loop
    acima_da_media = receitas[receitas > media]
    print(f"\n  Vendas acima da média: {len(acima_da_media)} de
{len(receitas)}")

    return {
        "media": media, "mediana": mediana,
        "desvio_padrao": desvio_padrao, "total": total
    }
```

RF08 – Criar Visualizações com Matplotlib e Seaborn

O projeto deverá gerar ao menos **3 gráficos distintos**, exportados como arquivos PNG. Os gráficos devem ser informativos, com título, rótulos de eixos e legenda onde aplicável.

Gráficos obrigatórios:

1. **Gráfico de linha:** Receita total por mês ao longo do tempo;
2. **Gráfico de barras:** Top 5 produtos ou categorias por receita;
3. **Gráfico adicional** (à escolha): dispersão, histograma, boxplot, mapa de calor (heatmap), pizza, ou barras agrupadas.

Exemplo:

Python

```
import matplotlib.pyplot as plt
import seaborn as sns
import os

def gerar_visualizacoes(df, metricas, output_dir="outputs/graficos"):
    """Gera e exporta visualizações dos dados de vendas."""
    os.makedirs(output_dir, exist_ok=True)

    # Configurações visuais globais
    sns.set_theme(style="whitegrid", palette="muted")
    plt.rcParams["figure.figsize"] = (12, 6)
    plt.rcParams["axes.titlesize"] = 14
    plt.rcParams["axes.labelsize"] = 12

    # --- Gráfico 1: Receita por Mês (linha) ---
    fig, ax = plt.subplots()
    por_mes = metricas["por_mes"]
    ax.plot(por_mes["mes"], por_mes["receita_total"], marker="o",
            linewidth=2, color="#2196F3")
    ax.fill_between(por_mes["mes"], por_mes["receita_total"], alpha=0.15,
                    color="#2196F3")
    ax.set_title("Receita Total por Mês (2024)")
    ax.set_xlabel("Mês")
    ax.set_ylabel("Receita Total (R$)")
    ax.set_xticks(range(1, 13))
    ax.set_xticklabels(["Jan", "Fev", "Mar", "Abr", "Mai", "Jun",
```

```

        "Jul", "Ago", "Set", "Out", "Nov", "Dez"],
rotation=45)
plt.tight_layout()
caminho = os.path.join(output_dir, "vendas_por_mes.png")
plt.savefig(caminho, dpi=150)
plt.close()
print(f" Gráfico exportado: {caminho}")

# --- Gráfico 2: Top 5 Produtos (barras horizontais) ---
fig, ax = plt.subplots()
top = metricas["top_produtos"]
sns.barplot(data=top, y="produto", x="receita_total", ax=ax,
palette="Blues_d")
ax.set_title("Top 5 Produtos por Receita Total")
ax.set_xlabel("Receita Total (R$)")
ax.set_ylabel("Produto")
for container in ax.containers:
    ax.bar_label(container, fmt="R$ %.0f", padding=5)
plt.tight_layout()
caminho = os.path.join(output_dir, "top_produtos.png")
plt.savefig(caminho, dpi=150)
plt.close()
print(f" Gráfico exportado: {caminho}")

# --- Gráfico 3: Distribuição de Receita por Região (boxplot) ---
fig, ax = plt.subplots()
sns.boxplot(data=df, x="regiao", y="receita_total", ax=ax,
palette="Set2")
ax.set_title("Distribuição de Receita por Transação – Por Região")
ax.set_xlabel("Região")
ax.set_ylabel("Receita por Venda (R$)")
plt.xticks(rotation=30)
plt.tight_layout()
caminho = os.path.join(output_dir, "distribuicao_regioes.png")
plt.savefig(caminho, dpi=150)
plt.close()
print(f" Gráfico exportado: {caminho}")

print("\n=== VISUALIZAÇÕES GERADAS COM SUCESSO ===")

```

RF09 – Criar uma Classe para o Pipeline

O estudante deverá criar ao menos **uma classe** que encapsule parte do pipeline de análise. A classe deve ter construtor (`__init__`), atributos e métodos que usem `self`.

Exemplo:

Python

```
class AnalisadorDeVendas:
    """
    Classe responsável por encapsular o pipeline de análise de vendas.
    Mantém o estado do DataFrame e os resultados intermediários.
    """

    def __init__(self, caminho_arquivo):
        """Inicializa o analisador com o caminho do arquivo de dados."""
        self.caminho_arquivo = caminho_arquivo
        self.df_bruto = None
        self.df_limpo = None
        self.metricas = {}
        self.clientes = None
        self.relatorio_limpeza = {}

    def carregar(self):
        """Lê o arquivo CSV e armazena o DataFrame bruto."""
        self.df_bruto = pd.read_csv(self.caminho_arquivo)
        print(f"[AnalisadorDeVendas] Arquivo carregado: {self.caminho_arquivo}")
        print(f"  Registros carregados: {len(self.df_bruto)}")
        return self

    def limpar(self):
        """Limpa os dados e armazena o DataFrame tratado."""
        self.df_limpo, self.relatorio_limpeza = limpar_dados(self.df_bruto.copy())
        return self

    def transformar(self):
        """Aplica transformações e cria colunas derivadas."""
        self.df_limpo = criar_colunas_derivadas(self.df_limpo)
```



```
        return self

def analisar(self):
    """Calcula métricas e segmentações."""
    self.metricas = calcular_metricas(self.df_limpo)
    self.clientes = segmentar_clientes(self.df_limpo)
    calcular_estatisticas_numpy(self.df_limpo)
    return self

def visualizar(self):
    """Gera e exporta os gráficos."""
    gerar_visualizacoes(self.df_limpo, self.metricas)
    return self

def exportar_relatorio(self, caminho="outputs/relatorio_resumo.csv"):
    """Exporta o relatório de métricas por mês em CSV."""
    os.makedirs("outputs", exist_ok=True)
    self.metricas["por_mes"].to_csv(caminho, index=False)
    print(f"\n[AnalisadorDeVendas] Relatório exportado: {caminho}")
    return self

def resumo(self):
    """Exibe um resumo executivo do pipeline."""
    print("\n" + "="*50)
    print("          RESUMO EXECUTIVO – SALESINSIGHT PY")
    print("="*50)
    print(f"  Arquivo analisado:      {self.caminho_arquivo}")
    print(f"  Registros brutos:      {self.relatorio_limpeza.get('registros_iniciais', 'N/A')}")
    print(f"  Registros limpos:      {self.relatorio_limpeza.get('registros_finais', 'N/A')}")
    receita = self.df_limpo["receita_total"].sum() if self.df_limpo is
not None else 0
    print(f"  Receita total anual:    R$ {receita:,.2f}")
    if self.clientes is not None:
        top = self.clientes.iloc[0]
        print(f"  Cliente top:           {top['cliente']} (R$ {top['total_gasto']:,.2f})")
    print("="*50)
```

RF10 – Usar Herança

O estudante deverá criar uma classe que herde de outra, adicionando funcionalidades específicas. Exemplo: uma classe `AnalizadorComProjecao` que herda de `AnalizadorDeVendas` e adiciona uma projeção simples de tendência.

Python

```
class AnalizadorComProjecao(AnalizadorDeVendas):
    """
    Extensão do AnalizadorDeVendas com funcionalidades de projeção
    simples.
    Herda todos os métodos da classe pai e adiciona projeção de tendência.
    """

    def __init__(self, caminho_arquivo, meses_projecao=3):
        super().__init__(caminho_arquivo)
        self.meses_projecao = meses_projecao
        self.projecoies = []

    def projetar_tendencia(self):
        """
        Projeta a receita dos próximos meses com base na média móvel dos
        últimos 3 meses.
        Método simples sem machine learning – baseado em médias.
        """
        if not self.metricas or "por_mes" not in self.metricas:
            print("[AVISO] Rode .analisar() antes de projetar.")
            return self

        por_mes = self.metricas["por_mes"].sort_values("mes")
        receitas_historicas = por_mes["receita_total"].to_numpy()

        # Média móvel dos últimos 3 meses como base da projeção
        ultimos_3 = receitas_historicas[-3:]
        media_movel = np.mean(ultimos_3)
        tendencia = np.std(ultimos_3) * 0.1 # fator de crescimento
        simples

        ultimo_mes = int(por_mes["mes"].max())

        print("\n=== PROJEÇÃO DE TENDÊNCIA (Média Móvel Simples) ===")
```

```
        print(f" Base: média dos últimos 3 meses = R$
{media_movel:,.2f}")
        self.projecoecoes = []

        for i in range(1, self.meses_projecao + 1):
            mes_projetado = (ultimo_mes + i - 1) % 12 + 1
            receita_projetada = media_movel + (tendencia * i)
            self.projecoecoes.append({"mes": mes_projetado,
"receita_projetada": round(receita_projetada, 2)})
            print(f" Mês {mes_projetado:02d} (projeção): R$
{receita_projetada:,.2f}")

        return self

    def exibir_projecao_detalhada(self):
        """Exibe as projeções calculadas."""
        if not self.projecoecoes:
            print("[AVISO] Nenhuma projeção disponível. Rode
.projetar_tendencia() primeiro.")
            return
        print("\n=== DETALHAMENTO DAS PROJEÇÕES ===")
        for p in self.projecoecoes:
            print(f" Mês {p['mes']:02d}: R$
{p['receita_projetada']:,.2f}")
```

RF11 – Usar Funções Lambda e Funções de Ordem Superior

O projeto deverá usar funções lambda em ao menos dois contextos distintos, e demonstrar uma função que recebe outra função como parâmetro (equivalente ao conceito de callback).

Exemplos de uso de lambda:

Python

```
# Lambda em apply (transformação condicional de coluna)
df["desconto"] = df["receita_total"].apply(lambda x: 0.10 if x > 10000
else 0.05)

# Lambda em sorted para ordenar lista de dicionários
```

```
produtos_ordenados = sorted(lista_produtos, key=lambda p: p["receita"],
reverse=True)

# Lambda como filtro rápido
vendas_alto_valor = df[df["receita_total"].apply(lambda x: x > 5000)]
```

Exemplo de função que recebe outra função como parâmetro:

Python

```
def processar_coluna(df, coluna, funcao_transformacao):
    """
    Aplica uma função de transformação a uma coluna do DataFrame.
    Demonstra o uso de funções como argumentos (higher-order function /
    callback).
    """
    df[f"{coluna}_transformado"] = df[coluna].apply(funcao_transformacao)
    print(f" Coluna '{coluna}_transformado' criada com sucesso.")
    return df

# Uso da função com lambda como callback
df = processar_coluna(df, "receita_total", lambda x: round(x / 1000, 2))
df = processar_coluna(df, "quantidade", lambda x: "Alto" if x > 5 else
"Baixo")
```

RF12 – Ler e Escrever Arquivos (CSV e JSON)

O projeto deverá demonstrar leitura e escrita de arquivos em pelo menos dois formatos.

Python

```
import json

def exportar_resultados(metricas, clientes, stats_numpy):
    """Exporta resultados em CSV e JSON."""
    os.makedirs("outputs", exist_ok=True)

    # Exportar CSV com métricas por mês
    caminho_csv = "outputs/metricas_por_mes.csv"
```

```
metricas["por_mes"].to_csv(caminho_csv, index=False,
encoding="utf-8-sig")
print(f" CSV exportado: {caminho_csv}")

# Exportar segmentação de clientes em CSV
caminho_clientes = "outputs/segmentacao_clientes.csv"
clientes.to_csv(caminho_clientes, index=False, encoding="utf-8-sig")
print(f" CSV exportado: {caminho_clientes}")

# Exportar estatísticas gerais em JSON
caminho_json = "outputs/estatisticas_gerais.json"
stats_serializaveis = {k: round(float(v), 2) for k, v in
stats_numpy.items()}
with open(caminho_json, "w", encoding="utf-8") as f:
    json.dump(stats_serializaveis, f, indent=4, ensure_ascii=False)
print(f" JSON exportado: {caminho_json}")

# Ler e exibir o JSON exportado para confirmar
with open(caminho_json, "r", encoding="utf-8") as f:
    dados_lidos = json.load(f)
print(f"\n Conteúdo do JSON exportado:\n {json.dumps(dados_lidos,
indent=2)}")
```

RF13 – Usar Expressões Regulares para Limpeza de Dados

O projeto deverá usar o módulo `re` para ao menos uma operação de limpeza ou validação de dados com expressões regulares.

Python

```
import re

def limpar_strings_com_regex(df):
    """
    Usa expressões regulares para limpeza de colunas de texto.
    Exemplos: remover caracteres especiais, padronizar formatos.
    """
```

```
# 1. Remover caracteres não alfanuméricos do nome do cliente (exceto
underline e espaço)
df["cliente_limpo"] = df["cliente"].apply(
    lambda s: re.sub(r"[^a-zA-Z0-9_ ]", "", str(s)).strip()
)

# 2. Identificar registros com padrão de ID inválido (deve ser
"Cliente_XXX")
padrao_cliente = re.compile(r"^Cliente_\d{3}$")
df["cliente_valido"] = df["cliente_limpo"].apply(
    lambda s: bool(padrao_cliente.match(s))
)

n_invalidos = (~df["cliente_valido"]).sum()
print(f"\n=== LIMPEZA COM REGEX ===")
print(f" Clientes com formato inválido encontrados: {n_invalidos}")
print(f" Amostra de clientes limpos:
{df['cliente_limpo'].head(5).tolist()}")

return df
```

RF14 – Executar o Pipeline Completo (Ponto de Entrada)

O projeto deverá ter um bloco principal que execute todo o pipeline de ponta a ponta, demonstrando o uso da classe e de todas as funções criadas.

Python

```
def main():
    """
    Função principal: executa o pipeline completo do SalesInsight PY.
    """
    print("\n" + "="*60)
    print("  SALESINSIGHT PY – Pipeline de Análise de Dados de Vendas")
    print("="*60)

    # Etapa 0: Gerar dataset (se necessário)
    if not os.path.exists("vendas.csv"):
        print("\n[INFO] Gerando dataset sintético...")
```

```
df_gerado = gerar_dataset_vendas(n_registros=200)
df_gerado.to_csv("vendas.csv", index=False)

# Etapa 1 a 6: Pipeline via classe com herança
analizador = AnalisadorComProjecao("vendas.csv", meses_projecao=3)
(analisador
 .carregar()
 .limpar()
 .transformar()
 .analisar()
 .projetar_tendencia()
 .visualizar()
 .exportar_relatorio()
)

# Etapa extra: limpeza com regex
analizador.df_limpo = limpar_strings_com_regex(analisador.df_limpo)

# Etapa extra: exportação JSON
stats = calcular_estatisticas_numpy(analisador.df_limpo)
exportar_resultados(analisador.metricas, analisador.clientes, stats)

# Resumo final
analizador.resumo()
analizador.exibir_projecao_detalhada()

print("\n[CONCLUÍDO] Pipeline finalizado com sucesso!")

if __name__ == "__main__":
    main()
```

4.3. REQUISITOS TÉCNICOS

Conteúdo	Obrigatório ?	Como Demonstrar
Kanban	Sim	Quadro com tarefas do projeto

Google Colab ou VS Code	Sim	Usar como ambiente de desenvolvimento
Extensões/ferramentas utilizadas	Sim	Citar no README
Como a internet funciona	Sim	Explicação curta no README
Arquitetura cliente-servidor	Sim	Menção no README (dados poderiam vir de uma API REST)
Versionamento	Sim	Commits no GitHub
GitHub	Sim	Repositório público
GitHub Desktop	Sim	Pode ser usado para commits e push
GitFlow simplificado	Sim	Branches simples com nomes descritivos
Lógica de programação	Sim	Cálculos, condicionais e repetições
Python	Sim	Arquivo .py principal
Tipos de dados	Sim	int, float, str, bool, list, dict
if / elif / else	Sim	Classificação de segmentos e faixas
Operadores aritméticos, lógicos e relacionais	Sim	Cálculo de receita, filtros, comparações
for / while	Sim	Loops de exibição ou iteração de dados
Funções com parâmetros e retorno	Sim	Todas as funções do pipeline
Funções lambda	Sim	Pelo menos 2 usos distintos
Função que recebe função como argumento	Sim	processar_coluna ou equivalente
Listas, dicionários e estruturas compostas	Sim	Dataset, metrics, relatório de limpeza
Leitura de CSV	Sim	pd.read_csv()
Escrita de CSV	Sim	.to_csv()
Escrita de JSON	Sim	json.dump()
Leitura de JSON	Sim	json.load()

Módulo datetime	Sim	Extração de mês, trimestre, ano
Expressões regulares (re)	Sim	Limpeza de strings com re.sub() / re.compile()
Pandas – Series e DataFrames	Sim	Estrutura principal de dados
Pandas – Leitura de fontes	Sim	read_csv() com tratamento
Pandas – Filtros e seleções	Sim	Filtros com condicionais booleanas
Pandas – groupby	Sim	Aggregações por mês, produto, região
Pandas – merge ou pivot	Opcional	Pode ser usado para enriquecer dados
NumPy – Arrays	Sim	Conversão de colunas e operações
NumPy – Operações vetorizadas	Sim	Sem loops em cálculos numéricos
NumPy – Broadcasting	Sim	Normalização ou comparação de arrays
NumPy – np.select	Sim	Transformação condicional vetorizada
Transformações condicionais	Sim	Classificação de faixas de receita
Matplotlib – gráfico de linha	Sim	Receita por mês
Matplotlib/Seaborn – gráfico de barras	Sim	Top produtos
Matplotlib/Seaborn – gráfico adicional	Sim	Boxplot, histograma, pizza ou dispersão
Exportação de gráficos (.png)	Sim	plt.savefig()
Classes com construtor (<code>__init__</code>)	Sim	AnalizadorDeVendas
Atributos de instância (<code>self</code>)	Sim	self.df_limpo, self.metricas etc.
Métodos de instância	Sim	.carregar(), .limpar(), .analisar()
Herança e <code>super()</code>	Sim	AnalizadorComProjecao extends AnalizadorDeVendas
Módulos e importação	Sim	import pandas, import numpy, etc.

4.4. ORGANIZAÇÃO DO KANBAN

O estudante ou squad deverá organizar o trabalho em um quadro Kanban.

Colunas obrigatórias: Backlog; A Fazer; Em Andamento; Concluído.

Pode usar: Trello; GitHub Projects; Notion; quadro simples no README; planilha; imagem do quadro.

Tarefas sugeridas para o Kanban:

- Criar repositório no GitHub
- Criar arquivo salesinsight.py
- Criar README.md
- Criar função para gerar ou carregar o dataset CSV
- Criar função de inspeção dos dados
- Criar função de limpeza de dados (nulos, datas inválidas, strings sujas)
- Criar função de transformação (colunas derivadas, datetime)
- Implementar groupby para métricas por mês, produto e região
- Usar NumPy para estatísticas vetorizadas
- Usar expressões regulares para limpeza de strings
- Criar função de segmentação de clientes com lambda
- Criar função de exportação (CSV + JSON)
- Criar classe AnalisadorDeVendas com construtor e métodos
- Criar classe AnalisadorComProjecao com herança e super()
- Criar função que recebe outra função como argumento
- Criar gráfico de linha (receita por mês) com Matplotlib
- Criar gráfico de barras (top produtos) com Seaborn
- Criar gráfico adicional (boxplot, histograma ou pizza)
- Exportar gráficos em PNG
- Criar função main() e bloco if __name__ == "__main__":
- Testar o pipeline completo no Colab ou VS Code
- Atualizar README com instruções e conceitos
- Fazer commits e criar branches no GitHub
- Gravar vídeo de demonstração
- Enviar links no AVA

4.5. VERSIONAMENTO COM GITHUB

O projeto deverá ter um repositório público no GitHub.

Branches mínimas:

Para projeto individual:

- main
- develop
- feat/pipeline-dados
- docs/readme

Para squad (até 03 pessoas):

- main
- develop
- feat/leitura-limpeza
- feat/analise-metricas
- feat/visualizacoes
- feat/classes-poo
- docs/readme-video

Commits mínimos:

- Mini-Projeto individual: **mínimo de 5 commits**
- Mini-Projeto em squad: **mínimo de 8 commits**

Exemplos de mensagens de commit (conventional commits):

Python

feat: cria estrutura inicial do projeto e arquivo salesinsight.py
feat: adiciona função de geração e leitura do dataset CSV
feat: implementa limpeza de dados e tratamento de nulos
feat: adiciona transformações de colunas e extração de datas
feat: implementa groupby para métricas por mês produto e região
feat: adiciona cálculos com NumPy e operações vetorizadas
feat: cria classe AnalisadorDeVendas com construtor e métodos
feat: cria AnalisadorComProjecao com herança e projeção de tendência
feat: adiciona visualizações com Matplotlib e Seaborn
feat: implementa exportação de CSV e JSON
docs: atualiza readme com instruções de execução e conceitos
fix: corrige conversão de datas inválidas no pipeline de limpeza

4.6. GRAVAÇÃO DE VÍDEO

Além do desenvolvimento das tarefas, você deverá gravar um vídeo (ou seu grupo de até 03 alunos, um vídeo por grupo), com tempo máximo de 5 minutos, abordando os seguintes tópicos:

- Qual o objetivo do sistema? Demonstre o pipeline rodando do início ao fim.
- O que é necessário instalar ou configurar para executar o projeto?
- Como você organizou as tarefas antes de começar a desenvolver? (Kanban)
- Quais branches você criou e qual o objetivo de cada uma?
- O que poderia ser melhorado no seu código ou no pipeline?

Você poderá gravar na vertical ou na horizontal. É importante que apareça seu rosto, que esteja em um local com boa iluminação e que seja possível ouvir sua voz com clareza. Para a entrega do vídeo, coloque-o em uma pasta do Google Drive com permissão de leitura para qualquer pessoa com o link (ou publique no YouTube como vídeo não listado) e compartilhe o link na submissão do AVA. Uma dica valiosa é inserir o link do vídeo diretamente no README.md do repositório no GitHub. É obrigatório que qualquer pessoa consiga acessar o vídeo pelo link fornecido.

5. CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto.

O mesmo possui variação de nota de 0 (zero) a 10 (dez) e possui peso de 25% sobre a avaliação do Módulo 01.

Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas.

Lembre-se: Você está livre para consultar materiais, documentação, exemplos e ferramentas de apoio, desde que consiga adaptar a solução ao desafio proposto e explicar o código entregue com segurança e confiança.

Apresentação do Projeto			
Nº	Critério de Avaliação	Parcial	Máximo
1	Estruturou o README.md contendo obrigatoriamente: objetivo do projeto, instruções de instalação e execução, lista de conceitos de Python e análise de dados aplicados, e links para o repositório ou vídeo.	0,75	1,0
2	Entregou o repositório no GitHub demonstrando o uso de branches descritivas para cada funcionalidade e	0,75	1,0

	um histórico de commits que reflete a evolução lógica do trabalho.		
3	Organizou o fluxo de desenvolvimento utilizando o quadro Kanban (Trello, GitHub Projects ou similar), mantendo a correlação direta entre as tarefas listadas e o código implementado.	0,75	1,0
4	Demonstrou fluidez técnica ao explicar, via vídeo, o funcionamento do pipeline, as decisões de implementação e as oportunidades de melhoria identificadas no projeto.	0,75	1,0

Desenvolvimento do Projeto

Nº	Critério de Avaliação	Pracial	Máximo
5	O projeto carregou, inspecionou e limpou corretamente o dataset, tratando nulos, datas inválidas e strings sujas, com uso de datetime e expressões regulares.	0,50	1,00
6	Criou colunas derivadas, calculou métricas agregadas com groupby, realizou segmentação de clientes com lambda e transformações condicionais com np.select.	0,75	1,50
7	Usou NumPy para operações vetorizadas, broadcasting e cálculo de estatísticas, demonstrando processamento sem loops sobre arrays.	0,40	0,75
8	Gerou ao menos 3 gráficos distintos e informativos com Matplotlib e/ou Seaborn, exportados como PNG, com títulos, rótulos e boa apresentação visual.	0,50	1,00
9	Aplicou POO com classe AnalisadorDeVendas (construtor, atributos, métodos com self) e herança em AnalisadorComProjecao com uso de super().	0,60	1,25
10	Usou funções lambda em contextos distintos, implementou função que recebe outra função como argumento, e leu/escreveu arquivos em CSV e JSON corretamente.	0,25	0,50

6. CHECKLIST FINAL DE ENTREGA

Antes de enviar no AVA, confira:

☐ Criei o repositório público no GitHub

☐ Criei o arquivo salesinsight.py

☐ Criei o README.md

- ☐ Criei o quadro Kanban
- ☐ Gerei ou incluí o dataset vendas.csv
- ☐ Inspecionei os dados com `.shape`, `.dtypes`, `.isnull()`
- ☐ Limpei os dados (nulos, datas inválidas, strings sujas)
- ☐ Usei o módulo `datetime` para extrair mês, trimestre e ano
- ☐ Usei expressões regulares (`re`) para limpeza de strings
- ☐ Criei colunas derivadas (`receita_total`, `trimestre`, `faixa_receita_item`)
- ☐ Usei `groupby` para agregar métricas por mês, produto e região
- ☐ Usei `np.select` ou equivalente para transformação condicional vetorizada
- ☐ Usei NumPy para estatísticas (`mean`, `std`, `median`, `percentile`)
- ☐ Demonstrei broadcasting ou operações vetorizadas com NumPy
- ☐ Usei funções `lambda` em pelo menos 2 contextos distintos
- ☐ Criei função que recebe outra função como argumento
- ☐ Exportei resultados em CSV com `to_csv()`
- ☐ Exportei resultados em JSON com `json.dump()`
- ☐ Li o JSON exportado com `json.load()` para confirmar
- ☐ Gerei gráfico de linha (receita por mês)
- ☐ Gerei gráfico de barras (top produtos ou categorias)
- ☐ Gerei um terceiro gráfico (boxplot, histograma, pizza ou dispersão)
- ☐ Exportei os gráficos em PNG com `plt.savefig()`
- ☐ Criei a classe `AnalizadorDeVendas` com `__init__`, atributos e métodos
- ☐ Usei `self` dentro dos métodos da classe
- ☐ Criei `AnalizadorComProjecao` herdando de `AnalizadorDeVendas` com `super()`
- ☐ Criei bloco `if __name__ == "__main__":` com função `main()`
- ☐ Fiz commits descritivos no GitHub
- ☐ Usei branches com nomes descritivos
- ☐ Gravei o vídeo de até 5 minutos

[] Coloquei o vídeo no Google Drive com permissão correta (ou YouTube não listado)

[] Enviei os links no AVA antes do prazo de entrega

7. MODELO DE README.md (SUGESTÃO)

Python

SrealesInsight PY

Sobre o projeto

O SalesInsight PY é um pipeline completo de análise de dados de vendas desenvolvido em Python.

O sistema lê, limpa, transforma e visualiza um dataset de vendas, gerando métricas, segmentações e projeções simples de tendência.

O que o sistema analisa

- Receita total e volume de vendas por mês e trimestre
- Top produtos e categorias por receita
- Desempenho por região
- Segmentação de clientes por nível de gasto (Bronze, Prata, Ouro)
- Projeção simples de tendência para os próximos meses
- Exportação de relatórios em CSV e JSON

Objetivo

Praticar os principais conceitos do Módulo 01 de IA para Análise Preditiva:

- Lógica de programação com Python
- Variáveis, tipos de dados e operadores
- Condicionais (if, elif, else) e repetição (for, while)
- Funções, parâmetros, retorno e funções lambda
- Funções de ordem superior (função que recebe função)
- Leitura e escrita de arquivos CSV e JSON
- Módulo datetime para manipulação de datas
- Expressões regulares com o módulo re
- Pandas: DataFrames, limpeza, groupby, filtros e transformações
- NumPy: arrays, operações vetorizadas, broadcasting, np.select

- Matplotlib e Seaborn: gráficos, customização e exportação em PNG
- Classes, construtores, atributos, métodos, herança e `super()`
- GitHub, branches, commits e GitFlow simplificado
- Kanban para organização do projeto

Como executar

No Google Colab (recomendado)

1. Faça upload do arquivo `salesinsight.py` e do `vendas.csv` para o Colab.
2. Execute: `!python salesinsight.py`
3. Ou cole o conteúdo diretamente em células de um notebook `.ipynb`.

Localmente com VS Code

1. Instale o Python 3.10+ e o VS Code.
2. Instale as dependências:

```
pip install pandas numpy matplotlib seaborn
```

Python

3. Execute no terminal:

```
python salesinsight.py
```

Python

Estrutura do projeto

```
salesinsight-py/
|
├── salesinsight.py      # Pipeline principal
├── vendas.csv           # Dataset de vendas (gerado pelo próprio
código ou externo)
├── README.md           # Este arquivo
├── outputs/
└── metricas_por_mes.csv
```



```
|— segmentacao_clientes.csv
|— estatisticas_gerais.json
|— graficos/
    |— vendas_por_mes.png
    |— top_produtos.png
    |— distribuicao_regioes.png
```

Ferramentas utilizadas

- Python 3.10+
- Google Colab / VS Code
- Bibliotecas: pandas, numpy, matplotlib, seaborn, re, json, datetime, os, random
- GitHub + GitHub Desktop para versionamento
- Trello / GitHub Projects para Kanban

Sobre var, let e const (Python equivalente)

Em Python, não existe a distinção entre `var`, `let` e `const` como no JavaScript.

Toda variável é declarada com simples atribuição. Por convenção, constantes são

escritas em MAIÚSCULAS (ex.: `CAMINHO\_ARQUIVO = "vendas.csv"`).

Para simular imutabilidade real, pode-se usar tuplas no lugar de listas.

Como a internet funciona (contexto do projeto)

Neste projeto, os dados são lidos de um arquivo local CSV. Em um cenário real de

produção, esses dados poderiam vir de uma API REST (ex.: uma requisição HTTP GET

para um servidor que retorna JSON). O cliente (seu script Python) faria a requisição,

o servidor processaria e retornaria os dados – seguindo a arquitetura cliente-servidor.

Bibliotecas como `requests` permitem consumir essas APIs diretamente no Python.

Vídeo de demonstração

[Inserir link do Google Drive ou YouTube aqui]

Documento elaborado para a disciplina de IA – Desenvolvimento de IA para Análise Preditiva. Estrutura adaptada do Mini-Projeto de Front-End React – Módulo 01.